Imbalanced Text Classification Study

GROUP 4
KOLASANI MAITHILI B230385CS
ARDHRA ALAINA WINSTON B230198CS
FARHEEN B230299CS
N SRITANVI B230069CS
GUNNA MOHANA PRATYUSHA B230324CS

INTRODUCTION

Text classification is one of the fundamental tasks in **Artificial Intelligence (AI)** and **Machine Learning (ML)**, where the goal is to assign predefined labels to textual data. It has many practical applications such as spam detection, sentiment analysis, and disaster tweet identification. In real-world scenarios, datasets are often imbalanced, meaning that one class has significantly more samples than the other.

This problem is known as the **class imbalance problem**, and it can reduce the effectiveness of machine learning models because the models tend to predict the majority class more often while performing poorly on the minority class. Therefore, handling class imbalance is an important step in building reliable and accurate classification systems.

In this project, we study the problem of imbalanced text classification using the **Disaster Tweets dataset**. The objective of this work is to implement multiple binary classification models and compare their performance before and after applying imbalance handling techniques.

The models used in this study are **Logistic Regression (LR)**, **Random Forest (RF)**, and a fixed-architecture **Neural Network (NN)**. All models are trained using **Term Frequency – Inverse Document Frequency (TF-IDF)** vectorized text features, which convert text data into numerical form suitable for machine learning algorithms.

The experiment is divided into two parts. In **Part A**, the models are trained on the original dataset without applying any imbalance handling technique. In **Part B**, imbalance handling methods such as oversampling, undersampling, and class weighting are applied to improve the performance of the models, especially for the minority class.

The performance of the models is evaluated using several metrics, including **Accuracy**, **Precision**, **Recall**, **F1-score**, **Confusion Matrix**, **Receiver Operating Characteristic – Area Under Curve (ROC-AUC)**, and **Precision–Recall Area Under Curve (PR-AUC)**. These evaluation measures help to better understand how class imbalance affects model performance and how different techniques can improve the results.

The main goal of this project is to analyze the impact of class imbalance on text classification models and to compare different imbalance handling methods in order to improve classification performance, particularly for the minority class.

Dataset Description

The dataset used in this project is the **Disaster Tweets dataset**, which is publicly available on Kaggle.
The dataset can be accessed from the following link:
Disaster Tweets

This dataset contains more than 11,000 tweets collected using disaster-related keywords such as *crash*, *quarantine*, *fire*, and other similar terms. Along with the tweet text, additional information such as the keyword and location associated with the tweet is also included. Each tweet was manually labeled to indicate whether it actually refers

to a real disaster event or not. In many cases, the same keywords may be used in different contexts, such as jokes, movie reviews, or casual conversations, which makes the classification task more challenging.

The structure of this dataset is based on the earlier **Disasters on Social Media** dataset, which was used for research on disaster detection from social media posts. The tweets were collected from real events such as natural disasters, accidents, and other emergency situations, and then manually classified into two categories:

- **Disaster (1)** – tweet refers to a real disaster event
- **Non-disaster (0)** – tweet does not refer to a real disaster event .This makes the dataset suitable for binary text classification problems.

In this project, only the tweet text and the target label are used for training the models. The dataset is slightly imbalanced, meaning that the number of tweets in one class is higher than the other, which makes it suitable for studying the effect of class imbalance on machine learning models.

Preprocessing

The tweet text was preprocessed before training the models in order to remove noise and convert the text into numerical features suitable for machine learning algorithms. The preprocessing steps are listed below:

- All tweet text was converted into string format to ensure consistency in processing.
- URLs and web links were removed using regular expressions.
- User mentions (such as @username) were removed from the text.
- Hashtag symbols (#) were removed while keeping the associated words.
- All non-alphabetic characters, including numbers and special symbols, were removed so that only letters and spaces remained.
- All text was converted to lowercase to maintain uniformity.
- Extra spaces were removed from the text.
- Tweets that became empty after cleaning were removed from the dataset.

Dataset splitting:

- The dataset was divided into training and testing sets using an 80–20 split.
- Stratified sampling was used to preserve the original class distribution in both training and testing data.

Feature extraction:

- The cleaned text was converted into numerical features using **Term Frequency – Inverse Document Frequency (TF-IDF)** vectorization.
- English stopwords were removed during vectorization.

- A maximum of 5000 features were used.
- Both unigram and bigram representations were included to capture contextual information.
- The TF-IDF feature vectors were used as input for all classification models.

Methodology

In this project, three different machine learning models were implemented to perform binary text classification on the Disaster Tweets dataset. All models were trained using TF-IDF feature vectors obtained from the preprocessed text. In Part A, the models were trained on the original dataset without applying any imbalance handling technique. The models used in this study are described below.

---

1. Logistic Regression

Logistic Regression is a linear classification algorithm commonly used for binary classification problems. It predicts the probability of a sample belonging to a particular class using a logistic function.

The following configuration was used:

- Regularization type: L2
- Regularization strength (C): 1.0
- Solver: liblinear
- Maximum iterations: 1000
- Decision threshold: 0.5

The model was trained using TF-IDF feature vectors obtained from the cleaned tweet text.

---

2. Random Forest

Random Forest is an ensemble learning method that combines multiple decision trees to improve classification performance and reduce overfitting. Each tree is trained on the TF-IDF features, and the final prediction is obtained by averaging the predictions of all trees.

The following configuration was used:

- Number of trees: 100
- Maximum depth: 20
- Minimum samples required to split: 5
- Minimum samples required at leaf node: 2

The Random Forest model was trained on the TF-IDF feature vectors generated from the tweet text.

---

3. Neural Network

A feed-forward Neural Network was implemented using PyTorch. The architecture was fixed as specified in the project instructions, and only the hyperparameters were tuned.

Network architecture:

- Input layer: TF-IDF feature vector
- Hidden layer 1: 64 neurons
- Hidden layer 2: 32 neurons
- Hidden layer 3: 16 neurons
- Output layer: 1 neuron with sigmoid activation

Training configuration:

- Activation function: ReLU
- Learning rate: 0.001
- Optimizer: Adam
- Batch size: 64
- Number of epochs: 20
- Dropout: 0.3
- L2 regularization: 1e-4
- Weight initialization: Xavier
- Decision threshold: 0.5

The neural network was trained using mini-batch gradient descent, and binary cross-entropy loss was used as the loss function.

Evaluation Metrics

- The performance of all models was evaluated using the following metrics:
- Accuracy
- Precision (per class)
- Recall (per class)
- F1-score (macro and weighted)
- Confusion Matrix
- ROC-AUC
- Precision–Recall curve
- PR-AUC

These metrics were used to compare the performance of the models and to observe the effect of class imbalance in Part A

Metric	Logistic Regression	Random Forest	Neural Network
Accuracy	0.8649	0.8289	0.8561
Precision (Class 0)	0.8665	0.8266	0.9163
Precision (Class 1)	0.8452	0.9722	0.6081
Recall (Class 0)	0.9859	0.9995	0.9059
Recall (Class 1)	0.3357	0.0827	0.6383
F1 Score (Macro)	0.7015	0.5287	0.7670
F1 Score (Weighted)	0.8402	0.7648	0.8575
ROC-AUC	0.8997	0.8420	0.8633
PR-AUC	0.7154	0.6305	0.6870

Handling Class Imbalance using Class Weighting

The dataset is imbalanced, with **9256 non-disaster samples** and **2114 disaster samples**.

- Class imbalance was addressed using **class weighting**, where higher importance is assigned to the minority class during training.
- This approach helps the model learn minority class patterns **without modifying the dataset size**.

Logistic Regression (Class Weighting)

Metric	Value
Accuracy	0.8478
Precision (Class 0)	0.9362
Precision (Class 1)	0.5701
Recall (Class 0)	0.8724
Recall (Class 1)	0.7400
F1 Macro	0.7736
F1 Weighted	0.8550
ROC-AUC	0.8999
PR-AUC	0.7124

Logistic Regression shows the **best overall performance**, with the **highest recall for the minority class (0.74)** and strongest ROC-AUC and PR-AUC values. It effectively balances precision and recall, making it the **most reliable model for detecting minority class instances**.

● ◇ Random Forest (Class Weighting)

Metric	Value
Accuracy	0.8500
Precision (Class 0)	0.8932
Precision (Class 1)	0.6158
Recall (Class 0)	0.9265
Recall (Class 1)	0.5154
F1 Macro	0.7353
F1 Weighted	0.8447
ROC-AUC	0.8436
PR-AUC	0.6263

Random Forest achieves high accuracy but has **poor recall for the minority class (0.5154)**, indicating a bias toward the majority class. It is **not suitable when detecting rare events is important**.

● ◇ Neural Network (Class Weighting)

Metric	Value
Accuracy	0.8355
Precision (Class 0)	0.9160
Precision (Class 1)	0.5491
Recall (Class 0)	0.8784
Recall (Class 1)	0.6478
F1 Macro	0.7456
F1 Weighted	0.8405
ROC-AUC	0.8512
PR-AUC	0.6600

The Neural Network provides a **balanced performance**, with better minority recall than Random Forest but lower than Logistic Regression. It offers a **good trade-off between precision and recall**, making it stable but not the best-performing model.

Class weighting significantly improves the ability of models to detect the minority class compared to the imbalanced baseline.

- Model performance still varies depending on the algorithm used.
- Logistic Regression performs the best due to:
- High recall for the minority class
- Strong ROC-AUC
- Highest macro F1-score
- Random Forest remains biased toward the majority class and struggles with minority detection.
- Neural Network provides a balanced performance but does not outperform Logistic Regression.
- **Final inference:** Class weighting is effective, but model selection is crucial, with Logistic Regression being the most suitable for this imbalanced text classification task.

Handling Class Imbalance using Random Sampling

- **Logistic Regression (Random Oversampling)**

Metric	Value
Accuracy	0.8407
Precision (Class 0)	0.9306
Precision (Class 1)	0.5560
Recall (Class 0)	0.8692
Recall (Class 1)	0.7163
F1 Macro	0.7624
F1 Weighted	0.8481
ROC-AUC	0.8930
PR-AUC	0.6993

Logistic Regression with oversampling achieves **high minority recall (0.7163)** similar to class weighting, with strong ROC-AUC and PR-AUC. However, it does not

outperform the class-weighted version, indicating that oversampling provides **no significant additional benefit for this model**.

Random Forest (Random Oversampling)

Metric	Value
Accuracy	0.8548
Precision (Class 0)	0.8885
Precision (Class 1)	0.6467
Recall (Class 0)	0.9395
Recall (Class 1)	0.4846
F1 Macro	0.7337
F1 Weighted	0.8464
ROC-AUC	0.8384
PR-AUC	0.6291

Random Forest still shows **low minority recall (0.4846)** even after oversampling, indicating that it **fails to effectively learn minority class patterns** and remains biased toward the majority class.

Neural Network (Random Oversampling)

Metric	Value
Accuracy	0.8663
Precision (Class 0)	0.9174
Precision (Class 1)	0.6413
Recall (Class 0)	0.9184
Recall (Class 1)	0.6383
F1 Macro	0.7788
F1 Weighted	0.8661
ROC-AUC	0.8685
PR-AUC	0.6740

The Neural Network performs the **best with oversampling**, achieving the highest accuracy and a well-balanced precision and recall. Oversampling significantly improves its ability to learn minority patterns, making it the **most effective model under this technique**.

Overall Inference (Oversampling)

- Oversampling balances the dataset and improves minority class learning.
- Logistic Regression shows **similar performance to class weighting**.
- Random Forest remains **weak in minority detection**.
- Neural Network benefits the most and becomes the **best-performing model with oversampling**.

Comparison Table – Part B (Balanced Dataset Techniques)

Model	Accuracy	Precision (C0)	Precision (C1)	Recall (C0)	Recall (C1)	F1 Macro	F1 Weighted
Class Weighting – Logistic Regression	0.8478	0.9362	0.5701	0.8724	0.7400	0.7736	0.8550
Class Weighting – Random Forest	0.8500	0.8932	0.6158	0.9265	0.5154	0.7353	0.8447
Class Weighting – Neural Network	0.8355	0.9160	0.5491	0.8784	0.6478	0.7456	0.8405
Oversampling – Logistic Regression	0.8407	0.9306	0.5560	0.8692	0.7163	0.7624	0.8481
Oversampling – Random Forest	0.8548	0.8885	0.6467	0.9395	0.4846	0.7337	0.8464
Oversampling – Neural Network	0.8663	0.9174	0.6413	0.9184	0.6383	0.7788	0.8661

🔹 Logistic Regression (Class Weighting vs Oversampling)

- Class Weighting:
  - Accuracy: 0.8478
  - Recall (Class 1): **0.7400**
  - F1 Macro: **0.7736**
  - ROC-AUC: **0.8999**
  - PR-AUC: **0.7124**
- Oversampling:
  - Accuracy: 0.8407
  - Recall (Class 1): 0.7163
  - F1 Macro: 0.7624
  - ROC-AUC: 0.8930

- PR-AUC: 0.6993

Class weighting performs slightly better across all key metrics.

Main Inference:
Logistic Regression works best with class weighting and provides the highest minority recall, making it the best model for detecting minority class instances.

🔹 Random Forest (Class Weighting vs Oversampling)

- Class Weighting:
  - Accuracy: 0.8500
  - Recall (Class 1): **0.5154**
  - F1 Macro: 0.7353
  - ROC-AUC: 0.8436
- Oversampling:
  - Accuracy: 0.8548
  - Recall (Class 1): 0.4846
  - F1 Macro: 0.7337
  - ROC-AUC: 0.8384

Oversampling slightly increases accuracy but reduces minority recall.

Main Inference:
Random Forest remains biased toward the majority class under both techniques and is the weakest model for minority detection.

🔹 Neural Network (Class Weighting vs Oversampling)

- Class Weighting:
  - Accuracy: 0.8355
  - Recall (Class 1): 0.6478
  - F1 Macro: 0.7456
  - PR-AUC: 0.6600
- Oversampling:
  - Accuracy: **0.8663**
  - Recall (Class 1): 0.6383
  - F1 Macro: **0.7788**
  - PR-AUC: **0.6740**

Oversampling significantly improves overall performance.

Main Inference:
Neural Network performs best with oversampling and becomes the strongest model in terms of overall balanced performance.

🔹 **Overall Comparison**

- Logistic Regression:
  - Best for **minority recall (0.7400)**
- Neural Network:
  - Best for **overall performance (accuracy & F1)**
- Random Forest:
  - Weak in minority detection

🔹 **Final Key Inferences**

- Class weighting works best for **Logistic Regression**
- Oversampling works best for **Neural Networks**
- Random Forest does not benefit significantly from either method

Final Conclusion:

- Use **Logistic Regression + Class Weighting** when minority detection is critical
- Use **Neural Network + Oversampling** for best overall balanced performance

Two Versions of the Implementation: With and Without GridSearchCV

The project requirements were ambiguous regarding whether hyperparameter tuning was expected as part of the implementation. To address this, two complete and independently runnable versions of the project were developed and submitted:

Version 1 — Without GridSearchCV (AIProjectFinal.ipynb, folder: WithoutGridSearchCV/): All model hyperparameters are set manually and kept fixed throughout the experiment. This version closely mirrors a standard baseline comparison study, where the focus is on the effect of imbalance-handling techniques rather than parameter optimization.

Version 2 — With GridSearchCV (AIprojectfinalWithGridSearchCV.ipynb, folder: WithGridSearchCV/): Hyperparameters for Logistic Regression and Random Forest are automatically selected using exhaustive grid search with cross-validation before the final models are trained and evaluated.

Both versions are fully functional and produce all required evaluation metrics. The results reported in this document correspond to the manually-tuned version (without

GridSearchCV). The GridSearchCV version is provided as a supplementary implementation to account for the possibility that automated hyperparameter tuning was intended by the project specification.

How GridSearchCV is Used

GridSearchCV is scikit-learn's built-in utility for exhaustive hyperparameter search combined with k-fold cross-validation. In the GridSearchCV version of this project, it is applied to **Logistic Regression** and **Random Forest** before training on both the baseline dataset (Part A) and the imbalance-handled variants (Part B). The Neural Network is excluded from GridSearchCV because its hyperparameters (learning rate, dropout, batch size, epochs) are tuned separately as PyTorch training arguments rather than scikit-learn estimator parameters.

The search is conducted as follows:

Logistic Regression parameter grid:

- C (regularization strength): [0.01, 0.1, 1.0, 10.0]
- penalty: ['l1', 'l2']
- solver: ['liblinear'] (compatible with both L1 and L2)

Random Forest parameter grid:

- n_estimators (number of trees): [50, 100, 200]
- max_depth: [10, 20, None]
- min_samples_split: [2, 5, 10]

Cross-validation is performed with **5-fold CV**, and the scoring metric used to select the best parameter combination is **F1-macro**, which is appropriate for imbalanced classification because it treats both classes equally when computing the average. The best estimator returned by GridSearchCV is then used for all subsequent training and evaluation steps, replacing the manually-specified hyperparameter values that appear in the non-GridSearchCV version.

This design ensures that the GridSearchCV version is a fair and rigorous comparison, where each model is given its best-possible configuration before being evaluated on the held-out test set, rather than relying on values chosen by manual inspection.

Summary: Key Differences Between the Two Versions

Hyperparameter selection: Without GridSearchCV uses fixed, manually specified values. With GridSearchCV uses automatically selected optimal values found via cross-validation.

Computation time: The GridSearchCV version is significantly slower due to the exhaustive search over parameter combinations and repeated cross-validation folds.

Reproducibility: Both versions use a fixed random seed (42) to ensure reproducible results across runs.

Scope: GridSearchCV is applied only to LR and RF. The Neural Network architecture and training procedure remain identical in both versions.

Note: Repository structure: The GitHub repository (https://github.com/Farheen-18/AIProject) organises the two versions into separate top-level folders (WithGridSearchCV/ and WithoutGridSearchCV/) so that either version can be run independently without interference.